Quest #74

Referred

MONEYFELLOWS

Hiring Quest – Senior Backend Engineer (.NET) @ MoneyFellows

Challenge-based hiring quest with structured evaluation and real project outcomes.

Status: Referred **Registration:** February 11, 2026 **Submission:** February 22, 2026

META INFO

CATEGORY	STATUS
Referred	Referred
REGISTRATION DEADLINE	**SUBMISSION DEADLINE**
February 11, 2026	February 22, 2026

PRIZE

Top performers get hired with a paid contract and the opportunity to work on real-world projects.

QUEST BRIEF

We are **MoneyFellows**, a fintech product company transforming how people save, plan, and manage money through secure, scalable digital financial solutions.
Our mission is to build reliable financial platforms that operate at scale while maintaining high standards of performance, security, and engineering excellence. (MoneyFellows)

We're hiring a **Senior Backend Software Engineer (.NET) (5+ YOE)** to join our backend team and help design, build, and optimize the core systems behind our products.

Start Date: Immediate

Salary: USD 1400 - 1900
Location: Hybrid – 5th Settlement, Cairo

How the Hiring Quest Works

- Register for the quest
- Receive the full challenge after registration closes
- Submit your solution before the deadline
- Top candidates are invited to a technical review session
- One candidate will be hired

Who We're Looking For

1. 5+ years of experience in .NET development or other OOP languages
2. Strong experience building and maintaining APIs, Microservices, and Databases
3. Solid understanding of Clean Architecture and modular system design
4. Hands-on experience with DDD (Domain Driven Design) concepts
5. Experience with Message Queues and Event Driven Architecture
6. Strong background in database profiling and optimization (RDBMS & NoSQL)
7. Familiar with CI/CD pipelines, cloud providers (AWS or Azure), and API security
8. Comfortable working with GitHub, EF Core, and modern development workflows

Bonus: A/B testing, serverless concepts, monitoring tools, fraud detection, SonarQube or similar code quality tools

---

Your Mission: "Event-Driven Financial Transactions Platform"

Business Context

MoneyFellows systems deal with **financial transactions**, where:

1. Services must be **independently deployable**
2. Data consistency must be handled safely
3. Events must never be lost or duplicated
4. The system must be extensible for future financial modules

In this quest, you'll design and implement a **mini event-driven backend platform** that reflects real-world fintech challenges.

The Challenge

🌊 Step 1 — Domain Modeling (DDD)

Model **two bounded contexts**:

A) Transactions Context

1. Create Transaction
2. Add Transaction Items
3. Submit Transaction
4. Cancel Transaction (with clear business rules)

B) Payments Context

1. Start Payment for a Transaction
2. Confirm Payment
3. Fail Payment

Requirements

1. Clear Aggregates and invariants
2. Domain Events such as:
   - TransactionSubmitted
   - PaymentConfirmed
   - PaymentFailed
3. Clean separation between bounded contexts

📄 Deliverable: DOMAIN.md explaining your domain decisions.

🌊 Step 2 — Microservices + Clean Architecture

Implement **two .NET services**:

Transactions Service

1. Endpoints:
    - POST /transactions
    - POST /transactions/{id}/items
    - POST /transactions/{id}/submit
    - POST /transactions/{id}/cancel

Payments Service

1. Endpoints:
    - POST /payments/start
    - POST /payments/{id}/confirm
    - POST /payments/{id}/fail

Architecture requirements

1. Clean Architecture layers (Domain / Application / Infrastructure / API)
2. No infrastructure dependencies leaking into domain
3. Explicit application use-cases

🌊 Step 3 — Event-Driven Workflow

Use a **message broker** (RabbitMQ preferred):

1. TransactionSubmitted → published by Transactions service
2. Payments service consumes event → creates payment intent
3. Payment result publishes:
    - PaymentConfirmed
    - PaymentFailed
4. Transactions service consumes events → updates transaction state

Reliability is key

🌿 Implement **Outbox Pattern** (or equivalent)
🌿 Idempotent event consumers
🌿 Basic retry & dead-letter handling

Step 4 — Observability & Testing

Add:

1. Correlation IDs across HTTP & events
2. Structured logging
3. Health checks
4. Tests:
    - Domain invariant tests
    - Integration test covering the full happy path

Suggested Tech Stack

1. **.NET 8 / ASP.NET Core**
2. **PostgreSQL** or **SQL Server**
3. **RabbitMQ**
4. **Docker Compose**
5. Optional frameworks: MassTransit, Wolverine, or NServiceBus (justify your choice)

Example Flow

- Create transaction
- Add items
- Submit transaction → event published
- Payment started
- Payment confirmed
- Transaction marked as completed

What You Should Submit

- **GitHub Repository**

1. /src/transactions-service

1. /src/transactions-service
2. /src/payments-service
3. docker-compose.yml
4. README.md
5. DOMAIN.md
6. Optional: ARCHITECTURE.md + diagram

10-Minute Video

1. **3 min:** Introduction + one complex backend challenge you solved
2. **7 min:** Live demo + architecture explanation + event reliability strategy

---

Evaluation Criteria

1. Architecture & DDD correctness – **25%**
2. Event-driven reliability – **25%**
3. Code quality & Clean Architecture – **20%**
4. Microservices boundaries & APIs – **15%**
5. Testing & observability – **10%**
6. Documentation – **5%**

Bonus: retries & DLQ, elegant domain modeling, clear trade-off explanations

---

After Submission

Top candidates will join a **technical review session** discussing:

1. Domain modeling decisions
2. Event consistency strategies
3. Scaling the system in real fintech production

Final decision within **5 business days** after the review.

QUEST SNAPSHOT

STATUS

Referred

REGISTRATION

February 11, 2026

SUBMISSION

February 22, 2026

Quest #74